



Discrete Audio Representations for Automated Audio Captioning

Jingguang Tian¹, Haoqin Sun^{1,2}, Xinhui Hu¹, Xinkang Xu¹

¹Hithink RoyalFlush AI Research Institute, Hangzhou, China

²TMCC, College of Computer Science, Nankai University, Tianjin, China

{tianjingguang, sunhaoqin, huxinhui, xuxinkang}@myhexin.com

Abstract

Discrete audio representations, termed audio tokens, are broadly categorized into semantic and acoustic tokens, typically generated through unsupervised tokenization of continuous audio representations. However, their applicability to automated audio captioning (AAC) remains underexplored. This paper systematically investigates the viability of audio token-driven models for AAC through comparative analyses of various tokenization methods. Our findings reveal that audio tokenization leads to performance degradation in AAC models compared to those that directly utilize continuous audio representations. To address this issue, we introduce a supervised audio tokenizer trained with an audio tagging objective. Unlike unsupervised tokenizers, which lack explicit semantic understanding, the proposed tokenizer effectively captures audio event information. Experiments conducted on the Clotho dataset demonstrate that the proposed audio tokens outperform conventional audio tokens in the AAC task.

Index Terms: automated audio captioning, audio tokens, discrete audio representations, supervised audio tokenizer

1. Introduction

Automated audio captioning (AAC) is a cross-modal translation task aimed at generating natural language descriptions for audio clips [1]. The primary approach employs an encoder-decoder-based deep learning framework, where the audio encoder extracts acoustic features, and the text decoder generates captions based on these features [2, 3, 4, 5, 6]. Recent studies frequently utilize pre-trained audio encoders, such as PANN [7], CNext [8], or BEATs [9], alongside language models like BERT [10], BART [11], and GPT-2 [12]. These methodologies encode rich information from audio signals into continuous representations. Although continuous representations effectively capture complex audio details, there is growing interest in discrete representations [13].

Audio tokenization transforms audio into discrete tokens, enabling seamless integration with large language models (LLMs). Audio tokens can be categorized into two types: acoustic and semantic tokens. Acoustic tokens are produced through neural compression (codecs) to reconstruct the original audio signal, ensuring it remains perceptually identical to listeners. Models like Soundstream [14], EnCodec [15], and DAC [16] are prominent neural compression models for acoustic tokenization, utilizing an encoder-decoder architecture and a residual vector quantizer (RVQ), where each layer quantizes residuals from the previous layer. In contrast, semantic tokens are derived through representation learning, which aims to maximize the preservation of the audio's semantic information. A common semantic tokenization approach involves selecting a

layer from a pre-trained self-supervised learning (SSL) or supervised learning (SL) model and clustering its representations, often using the k-means algorithm, as seen in HuBERT [17] and WavLM [18]. RepCodec [19] is a tokenization method that employs parametric networks to extract semantic tokens. Unlike conventional audio codecs that reconstruct the original audio, RepCodec focuses on reconstructing audio representations. However, these audio tokens are typically obtained through unsupervised learning, which inspires an idea: whether they can be further enhanced through supervised training.

Numerous studies have explored the potential of discrete audio tokens as an alternative to continuous representations, particularly in speech-related tasks. The paradigm of LLM-based speech generation, which conditions on text to model audio tokens, has gained prominence due to its ability to produce highly natural-sounding voices [20, 21]. Discrete representations have also demonstrated competitive performance in various discriminative tasks [13, 22, 23, 24], including speech recognition, speech translation, speaker recognition, speaker diarization, and speech emotion recognition. However, the application of discrete audio representations in AAC is still in its early stages. EnCLAP [25] has shown that combining EnCodec with continuous audio representations provides a more effective input for pre-trained language models by leveraging the strengths of both discrete and continuous representations. Despite these advancements, the use of audio tokens in AAC systems remains largely unexplored.

In this paper, we explore the integration of discrete audio representations into two representative models, BART Base and GPT-2 XL, for the development of AAC systems. We employ pre-trained BEATs to extract continuous audio representations, which are then discretized using k-means and RepCodec to generate semantic tokens. For acoustic tokens, we utilize EnCodec and DAC. Our comparative analysis reveals that semantic tokens significantly outperform acoustic tokens in the AAC task. However, despite these advancements, we observe that audio discretization degrades the performance of AAC models compared to models that directly utilize continuous representations. To mitigate this issue, we propose a novel approach for deriving semantic tokens through supervised learning. This involves integrating a vector quantization module into pre-trained BEATs and training the model with a multi-label audio tagging loss. Experimental evaluations conducted on the Clotho dataset demonstrate that AAC models utilizing the proposed semantic tokens achieve performance levels that are competitive with those employing continuous representations.

The remainder of the paper is organized as follows: Section 2 outlines the methodology for constructing AAC systems utilizing discrete audio representations. Section 3 presents the experimental results. Finally, Section 4 concludes the paper.

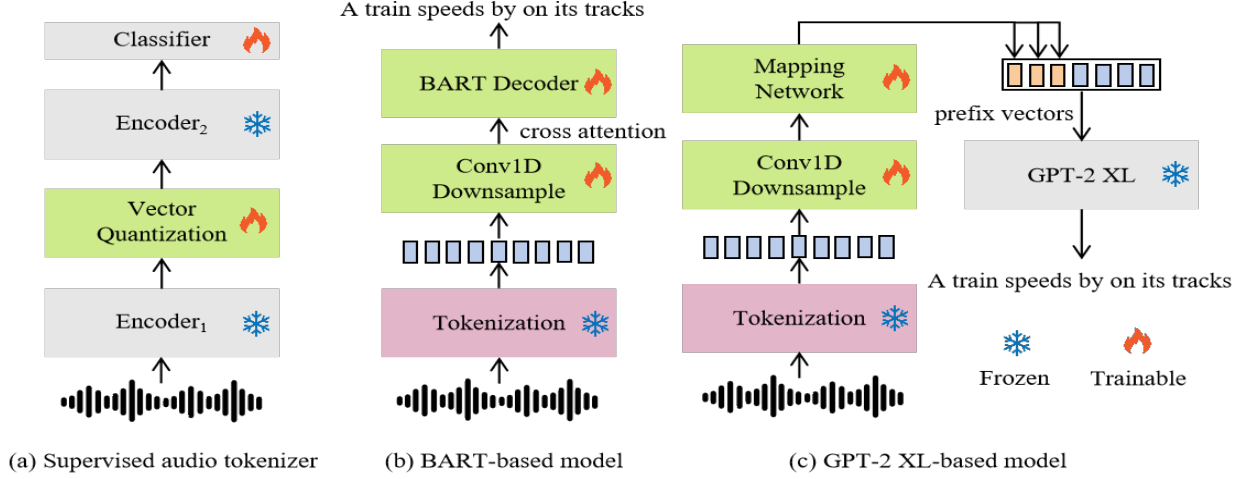


Figure 1: Schematic diagram of the proposed automated audio captioning models utilizing audio tokens.

2. Method

This section describes the AAC models that utilize audio tokens, as illustrated in Figure 1. We construct BART-based and GPT-2 XL-based AAC systems, both utilizing semantic and acoustic tokens.

2.1. Acoustic tokens

2.1.1. EnCodec

EnCodec is an audio codec based on a convolutional encoder-decoder architecture trained in an end-to-end manner. We employ the pre-trained EnCodec¹ to process audio at a bitrate of 24 kbps, utilizing 32 codebooks, each containing 1024 code vectors with 128 dimensions. The model transforms 24 kHz audio into discrete token sequences at a frame rate of 75 Hz.

2.1.2. DAC

DAC, an enhanced version of EnCodec, combines advancements in high-fidelity audio generation with improved codebook learning techniques, alongside refined adversarial and reconstruction losses. We use a pre-trained DAC² with 32 codebooks, each containing 1024 code vectors of size 8, to compress 24 kHz audio into discrete tokens at a frame rate of 75 Hz.

2.2. Semantic tokens

2.2.1. BEATs

BEATs, a state-of-the-art audio tagging model, is trained using an iterative audio pretraining framework. It transforms 16 kHz audio waveforms into 128-dimensional Mel filter bank features (Fbank) with a 10 ms frame shift. These features are segmented into 16×16 patches and converted into patch embeddings via a linear projection layer, producing 50 patch embeddings per second of audio. The embeddings are then processed by 12 Transformer encoder layers with 768-dimensional hidden states to generate encoded audio representations. We use the output from the ninth layer of the pre-trained BEATs³ as continuous audio representations and apply various tokenization methods to obtain semantic tokens.

¹<https://github.com/facebookresearch/encodec>

²<https://github.com/descriptinc/descript-audio-codec>

³<https://github.com/microsoft/unilm/tree/master/beats>

2.2.2. K-means

K-means clustering is a widely used method for semantic token extraction. We first extract continuous audio representations from the training data using BEATs, then determine the number of clusters, K , fit the k-means model (codebook), and obtain K cluster centroids (code vectors). During the tokenization process, the Euclidean distance between the continuous audio representations and the cluster centroids is calculated. The centroid index with the smallest distance is selected as the discretized token.

2.2.3. RepCodec

RepCodec employs an end-to-end neural codec to more effectively preserve information within continuous audio representations. It consists of an encoder, a vector quantizer (VQ) [26], and a decoder. The VQ compresses the encoder’s output representations into discrete tokens by projecting them onto their closest codebook entries, which are then passed to the decoder to reconstruct the input representations. In this work, we utilize BEATs to extract continuous audio representations for reproducing RepCodec. More details can be found in [19].

2.2.4. Supervised audio tokenizer

Our proposal introduces a supervised audio tokenizer designed to explicitly capture semantic information. This approach contrasts with prevalent audio tokenizers, which are predominantly developed through unsupervised learning methods and thus lack the capacity to acquire explicit knowledge of audio content. Transferring audio event knowledge from audio tagging models—which recognize specific sound events within an audio recording—to AAC results in substantial performance improvements [27]. We hypothesize that a tokenizer trained with audio tagging objectives enhances semantic representation, thereby benefiting AAC. Advanced audio tagging models often undergo a multifaceted process that includes self-supervised pre-training followed by supervised fine-tuning. Rather than training from scratch, we employ a more computationally efficient approach.

Specifically, we split the pre-trained BEATs model into two parts: the first 9 layers form *Encoder₁*, and the last 3 layers form *Encoder₂*. A vector quantization module is then incorporated between the two. The network architecture of this vector quantization module mirrors that of RepCodec [19]. During the

training stage, as illustrated in Figure 1 (a), the parameters of $Encoder_1$ and $Encoder_2$ are fixed. Codebook embeddings are updated using an exponentially moving average (EMA), while the encoder and decoder of the vector quantization module are optimized via binary cross-entropy loss. We denote the audio as a_n , where n is the index of the audio files, and $f(a_n) \in [0, 1]^S$ represents the model’s output probabilities for S sound classes. The label of a_n is denoted as $l_n \in \{0, 1\}^S$. The binary cross-entropy loss function L_B is defined as:

$$L_B = - \sum_{n=1}^N (l_n \cdot \ln f(a_n) + (1 - l_n) \cdot \ln (1 - f(a_n))) \quad (1)$$

where N is the number of training samples. It is worth noting that our supervised tokenizer training scheme is adaptable to any pre-trained audio tagging model. Empirical evidence shows that selecting higher layers for splitting audio tagging models yields better results. After training, $Encoder_1$, the encoder of the vector quantization module, and the vector quantizer form a pipeline for converting audio waveforms into semantic tokens.

2.3. AAC models

2.3.1. BART-based model

The BART-based AAC model employs an encoder-decoder architecture, as shown in Figure 1 (b). The encoder utilizes a single-layer 1D convolution with a kernel size of 3 and a stride of 3, primarily aimed at achieving a threefold reduction in the input sequence length. A 6-layer BART Base Transformer decoder is utilized for caption generation, with the pretrained BART⁴ text tokenizer applied during training and the decoder weights initialized randomly. The BART decoder cross-attends to the encoder’s output while self-attending to previous caption tokens in an autoregressive manner. Given the input audio representations $\mathbf{x} = x_1, x_2, \dots, x_n$, and their corresponding captions $\mathbf{y} = y_1, y_2, \dots, y_m$, the model’s training objective is to minimize cross-entropy loss:

$$L_{BART} = - \frac{1}{m} \sum_{t=1}^m \log p(y_t | y_{1:t-1}, \mathbf{x}) \quad (2)$$

where y_t is the t^{th} token in the caption.

2.3.2. GPT-2 XL-based model

The GPT-2 XL-based AAC model employs a decoder-only framework, as shown in Figure 1 (c), inspired by the work of [28]. The input representation sequence undergoes a $3 \times$ downsampling via a Conv1D layer, analogous to the BART-based model. To handle varying audio representation lengths, we concatenate a fixed number of learnable embeddings with the downsampled features and feed them into a mapping network consisting of a single Transformer layer with 8 heads and an embedding dimension of 1,600. From the output embeddings of the mapping network, we discard the portion corresponding to the downsampled features and retain only the portion corresponding to the learnable embeddings as prefix vectors for GPT-2 XL. Subsequently, the GPT-2 XL model generates captions conditioned on prefix vectors $\mathbf{p} = p_1, p_2, \dots, p_k$, where k is the prefix length. In our experiments, k is set to 50. During training, the pre-trained GPT-2 XL⁵ model remains frozen, while the Conv1D and mapping network are updated to learn prefix mappings comprehensible to the GPT-2 XL. The model

is trained by cross-entropy loss, calculated solely for the tokens corresponding to the captions:

$$L_{GPT-2 XL} = - \frac{1}{m} \sum_{t=1}^m \log p(y_t | \mathbf{p}, y_{1:t-1}) \quad (3)$$

3. Experiment

This section provides a detailed description of the datasets, experimental setup, and evaluation metrics used in our study.

3.1. Datasets

AudioSet [29] is an audio dataset comprising 527 sound classes and approximately 2 million 10-second audio clips. The dataset is divided into three subsets: an unbalanced subset containing 2,042,985 clips, a balanced subset with 22,176 clips, and an evaluation set comprising 20,383 clips. We utilize the AudioSet dataset to train audio tokenizers for extracting semantic tokens.

Clotho [30] consists of 15 to 30-second audio clips, each paired with five captions containing 8 to 20 words. For our experiments, we use the Clotho v2 dataset, which comprises 3,839 development samples, 1,045 validation samples, and 1,045 evaluation samples. The development set is used to train the AAC models, the validation set for parameter tuning, and the evaluation set to assess the model’s performance.

3.2. Experimental setup

3.2.1. Tokenizer model

We classify Clotho as in-domain data and AudioSet as out-of-domain data to investigate the domain effect on the tokenizer. The k-means model is trained on the Clotho development set and the balanced subset of AudioSet, with K set to 1,024. The RepCodec model is trained on the Clotho development set and the unbalanced subset of AudioSet. Due to the absence of sound labels in Clotho, the supervised audio tokenizer is exclusively trained on the unbalanced subset of AudioSet. For RepCodec and the supervised audio tokenizer, we employ two types of quantizers—conventional VQ [26] and RVQ [14]—both utilizing codebooks of size 1,024. Notably, RVQ uses two codebooks; with one, it is equivalent to VQ. Both models are trained for 100K steps with a batch size of 64, using the AdamW optimizer and a constant learning rate of $1e-5$.

During the discretization process, we forgo using quantized audio codes, opting instead for the corresponding code vectors as discrete audio representations, due to their slight performance advantage. For tokenizers with multiple codebooks, the corresponding code vector sequences are summed to form the final quantized representation.

3.2.2. AAC model

We explore five discrete audio representations and employ two continuous audio representations (FBANK and BEATs) for comparative analysis in building AAC models. All input audio representations are extracted from 30-second segments, with zero-padding applied to shorter audio clips. Since different input audio representations have varying dimensions, the input dimension of Conv1D is adjusted accordingly. Captions are converted to lowercase, and punctuation is removed. The model is trained for 20 epochs on the Clotho development set with a batch size of 16, using the AdamW optimizer, an initial learning rate of $2e-5$ with linear decay, and a weight decay of $3e-4$. During inference, beam search with a beam size of 3 is utilized.

⁴<https://huggingface.co/facebook/bart-base>

⁵<https://huggingface.co/openai-community/gpt2-xl>

Table 1: The performance of AAC models on the Clotho evaluation set. RepCodec-VQ/RVQ and Supervised-VQ/RVQ denote the RepCodec and supervised audio tokenizer with VQ/RVQ, respectively. ID indicates training the tokenizer with in-domain data, while OD indicates training with out-of-domain data. * indicates that the results are sourced from [25].

Tokenizer	BART-based AAC model						GPT-2 XL-based AAC model						ENCLAP*
	SPIDeR (↑)		FENSE (↑)		#Words (↑)		SPIDeR (↑)		FENSE (↑)		#Words (↑)		SPIDeR (↑)
	ID	OD	ID	OD	ID	OD	ID	OD	ID	OD	ID	OD	
K-means	0.285	0.267	0.483	0.474	436	457	0.256	0.256	0.474	0.463	725	757	0.294
RepCodec-VQ	0.288	0.275	0.483	0.471	459	440	0.265	0.256	0.471	0.469	561	715	
RepCodec-RVQ	0.294	0.292	0.496	0.487	475	488	0.279	0.268	0.485	0.477	692	658	
Supervised-VQ	—	0.285	—	0.479	—	425	—	0.268	—	0.477	—	684	
Supervised-RVQ	—	0.294	—	0.490	—	474	—	0.277	—	0.484	—	693	
EnCodec	—	0.085	—	0.217	—	98	—	0.129	—	0.284	—	448	
DAC	—	0.143	—	0.322	—	235	—	0.154	—	0.324	—	487	
FBANK	0.125		0.280		152		0.076		0.214		45		
BEATs	0.299		0.503		493		0.270		0.486		837		

3.3. Metrics

To evaluate the performance of the models, we employ SPIDeR [31], FENSE [32], and the unique word vocabulary of captions (referred to as #Words). While SPIDeR, though valuable for comparative analysis with existing literature, exhibits high sensitivity to n-gram matching, FENSE is a metric based on two pre-trained models that compares sentence embeddings rather than n-grams. The unique word vocabulary provides insights into the diversity of the model’s outputs.

3.4. Results

We present the results on the Clotho evaluation set in Table 1. EnCodec and DAC perform significantly worse than k-means, RepCodec, and the proposed supervised audio tokenizer, as they are designed to reconstruct original audio, focusing on acoustic details while neglecting crucial semantic information. BEATs features, rich in high-level semantics, are effectively discretized by k-means, RepCodec, and the proposed supervised audio tokenizer, preserving their semantic information and making them more suitable for AAC. The bottom row of Table 1 shows that using continuous audio representations from BEATs in AAC models outperforms using discrete representations from k-means, RepCodec, and the proposed supervised audio tokenizer. This indicates that discretizing audio representations leads to information loss. However, the proposed supervised audio tokenizer achieves the best performance among tokenization methods on out-of-domain training data, nearly matching the performance of continuous audio representations. This suggests that supervised learning with audio tagging enables the tokenizer to capture knowledge of audio events, thereby mitigating information loss during discretization.

Further analysis reveals that the GPT-2 XL-based AAC model underperforms compared to the BART-based model in both SPIDeR and FENSE metrics, although it generates more diverse captions. Given Clotho’s limited size, we hypothesize that the GPT-2 XL-based AAC model is more prone to overfitting in low-resource settings, despite retaining the robust language expressivity of the LLM. In most scenarios, the AAC model using discrete audio representations outperforms the FBANK-based model. Moreover, our BART-based model, which uses only discrete audio representations, performs comparably to the ENCLAP model that leverages both discrete and continuous audio representations. These findings highlight the significant potential of discrete audio representations for AAC.

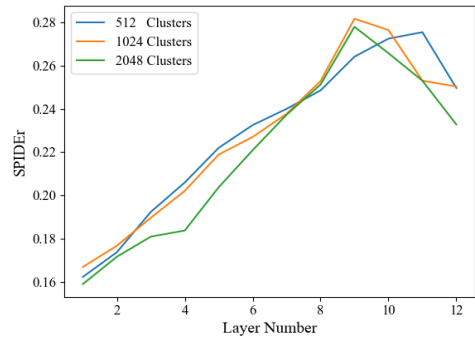


Figure 2: Performance of k-means applied to the BART-based AAC model across different layers and cluster numbers.

3.5. Ablation studies

Employing k-means clustering as a case study, we investigate how layer selection within the BEATs model and codebook size (number of clusters) affect semantic token performance, as depicted in Figure 2. The results indicate that features from the upper layers are more suitable for AAC, and the optimal number of layers and clusters is interdependent. Additionally, we explore several pivotal factors, including the influence of training data domains and the number of codebooks used by the quantizer for generating semantic tokens, as illustrated in Table 1. For k-means and RepCodec, in-domain training data outperforms out-of-domain data. Moreover, for RepCodec and the proposed supervised audio tokenizer, increasing the number of codebooks in the quantizer enhances performance.

4. Conclusion

This paper presents the first systematic study of discrete audio representations for AAC. Our findings indicate that semantic tokens significantly outperform acoustic tokens in this context. Moreover, the discretization of continuous audio representations leads to information loss, diminishing AAC performance. To mitigate this, we develop a supervised audio tokenizer to extract semantic tokens, leveraging an audio tagging objective for training. This method incorporates explicit semantic knowledge to compensate for the information loss due to discretization. Experimental results show that the proposed supervised audio tokenizer surpasses existing methods and achieves comparable results to continuous representations in the AAC task.

5. References

- [1] X. Mei, X. Liu, M. D. Plumbley, and W. Wang, "Automated audio captioning: An overview of recent progress and new challenges," *EURASIP journal on audio, speech, and music processing*, vol. 2022, no. 1, p. 26, 2022.
- [2] K. Chen, Y. Wu, Z. Wang, X. Zhang, F. Nian, S. Li, and X. Shao, "Audio captioning based on transformer and pre-trained cnn," in *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2020 Workshop (DCASE2020)*, 2020, pp. 21–25.
- [3] F. Gontier, R. Serizel, and C. Cerisara, "Automated audio captioning by fine-tuning bart with audioset tags," in *DCASE 2021-6th Workshop on Detection and Classification of Acoustic Scenes and Events*, 2021.
- [4] X. Liu, X. Mei, Q. Huang, J. Sun, J. Zhao, H. Liu, M. D. Plumbley, V. Kilic, and W. Wang, "Leveraging pre-trained bert for audio captioning," in *2022 30th European Signal Processing Conference (EUSIPCO)*. IEEE, 2022, pp. 1145–1149.
- [5] E. Labb, T. Pellegrini, J. Pinquier *et al.*, "Conette: An efficient audio captioning system leveraging multiple datasets with task embedding," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.
- [6] S.-L. Wu, X. Chang, G. Wichern, J.-w. Jung, F. Germain, J. Le Roux, and S. Watanabe, "Improving audio captioning models with fine-grained audio features, text embedding supervision, and llm mix-up augmentation," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 316–320.
- [7] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "Panns: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.
- [8] T. Pellegrini, I. Khalfaoui-Hassani, E. Labbé, and T. Masque-lier, "Adapting a convnext model to audio classification on audioset," in *24th INTERSPEECH Conference (INTERSPEECH 2023)*. ISCA, 2023, pp. 4169–4173.
- [9] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, W. Che, X. Yu, and F. Wei, "Beats: audio pre-training with acoustic tokenizers," in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 5178–5193.
- [10] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [11] M. Lewis, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.
- [12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [13] S. Vashishth, H. Singh, S. Bharadwaj, S. Ganapathy, C. Asawaroengchai, K. Audhkhasi, A. Rosenberg, A. Bapna, and B. Ramabhadran, "Stab: Speech tokenizer assessment benchmark," *arXiv preprint arXiv:2409.02384*, 2024.
- [14] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.
- [15] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.
- [16] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved rvqgan," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [17] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM transactions on audio, speech, and language processing*, vol. 29, pp. 3451–3460, 2021.
- [18] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [19] Z. Huang, C. Meng, and T. Ko, "RepCodec: A speech representation codec for speech tokenization," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 5777–5790.
- [20] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li *et al.*, "Neural codec language models are zero-shot text to speech synthesizers," *arXiv preprint arXiv:2301.02111*, 2023.
- [21] M. Łajszczak, G. Cámbara, Y. Li, F. Beyhan, A. van Korlaar, F. Yang, A. Joly, Á. Martín-Cortinas, A. Abbas, A. Michalski *et al.*, "Base tts: Lessons from building a billion-parameter text-to-speech model on 100k hours of data," *arXiv preprint arXiv:2402.08093*, 2024.
- [22] X. Chang, B. Yan, Y. Fujita, T. Maekaku, and S. Watanabe, "Exploration of efficient end-to-end asr using discretized input from self-supervised learning," in *INTERSPEECH 2023*, 2023, pp. 1399–1403.
- [23] P. Mousavi, J. Duret, S. Zaiem, L. Della Libera, A. Ploujnikov, C. Subakan, and M. Ravanelli, "How should we extract discrete audio tokens from self-supervised models?" in *Interspeech 2024*, 2024, pp. 2554–2558.
- [24] K. C. Puvvada, N. R. Koluguri, K. Dhawan, J. Balam, and B. Ginsburg, "Discrete audio representation as an alternative to mel-spectrograms for speaker and speech recognition," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 12 111–12 115.
- [25] J. Kim, J. Jung, J. Lee, and S. H. Woo, "Enclap: Combining neural audio codec and audio-text joint embedding for automated audio captioning," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 6735–6739.
- [26] A. Van Den Oord, O. Vinyals *et al.*, "Neural discrete representation learning," *Advances in neural information processing systems*, vol. 30, 2017.
- [27] X. Xu, H. Dinkel, M. Wu, Z. Xie, and K. Yu, "Investigating local and global information for automated audio captioning with transfer learning," in *ICASSP 2021-2021 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2021, pp. 905–909.
- [28] M. Kim, K. Sung-Bin, and T.-H. Oh, "Prefix tuning for automated audio captioning," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [29] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio set: An ontology and human-labeled dataset for audio events," in *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2017, pp. 776–780.
- [30] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An audio captioning dataset," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 736–740.
- [31] S. Liu, Z. Zhu, N. Ye, S. Guadarrama, and K. Murphy, "Improved image captioning via policy gradient optimization of spider," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 873–881.
- [32] Z. Zhou, Z. Zhang, X. Xu, Z. Xie, M. Wu, and K. Q. Zhu, "Can audio captions be evaluated with image caption metrics?" in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 981–985.